

NevIR: Negation in Neural Information Retrieval

Orion Weller, Dawn Lawrie, Benjamin Van Durme

Report by

Abdul Rafay Mughal | 23K-0667

Muhammad Hammad | 23I-0544

Abdullah Azhar Khan | 23K-0691

Table of Contents

Abstract	2
Introduction	2
Background	2
Neural Information Retrieval	2
Negation in IR	3
Overview of the Original Paper	3
Paper Objective	3
Paper Objective	3
Evaluation Metric Used in the Paper	4
Original Paper Results	4
Our Implementation	5
Repository Overview	5
Dataset Used	5
Models Implemented	5
Experimental Setup	5
Evaluation Metrics	6
Query Accuracy	6
Pairwise Accuracy	6
Mean Reciprocal Rank	6
Our Results	7
Validation Results	7
Test Results	7
Comparison With the Original Paper	7
Why Our Results Differ From the Paper	8
Different Model Coverage	8
Different Checkpoints	8
Preprocessing Differences	9
Tie Handling	9
Metric Differences	9
Dataset Split Naming/Version Differences	9
MonoT5 Implementation Details	9
Error Analysis	9
Discussion	10
Limitations	10
Conclusion	10
Figures	11
On Validation Split	11
On Test Split	12

Abstract

This project reproduces the study NevIR: Negation in Neural Information Retrieval, which evaluates whether information retrieval models correctly handle negation in queries and documents. The original paper introduces NevIR, a contrastive benchmark where each example contains two similar documents and two corresponding queries, with the correct ranking depending on negation understanding. In our implementation, we evaluated retrieval models including Random, BM25, TF-IDF, Sentence-BERT, and MonoT5 on the NevIR dataset. Our results follow the general trend reported in the paper: lexical and embedding-based models perform poorly on negation-sensitive retrieval, while MonoT5 performs significantly better. However, our scores do not exactly match the paper because our implementation uses a smaller subset of models, different model checkpoints, and slightly different preprocessing/evaluation choices.

Introduction

Negation is common in natural language, but neural language models and retrieval systems often fail to understand it correctly. In information retrieval, this is serious because a system may retrieve a passage that is lexically similar to the query but semantically opposite. The NevIR paper motivates this with examples where missing words such as “not,” “except,” or “without” changes the meaning of a query or document. The paper argues that this weakness is especially important for high-stakes domains such as medicine, law, education, and search engines.

The goal of our project was to reproduce the NevIR evaluation setup, implement several retrieval baselines, compute the same or related metrics, and compare our results with those reported in the original paper.

Background

Neural Information Retrieval

Information retrieval is the task of finding documents relevant to a user query. Traditional methods such as TF-IDF and BM25 rely mainly on lexical matching, while neural IR models use learned language representations to capture semantic similarity. However, semantic similarity alone may not be enough for negation because two sentences can be lexically and semantically close while having opposite meanings.

Negation in IR

Negation creates a difficult retrieval case because the relevant and irrelevant documents may share almost all words. A model that relies only on surface overlap or broad semantic similarity may rank both documents similarly, or may prefer the same document for both the negated and non-negated query. The NevIR paper states that prior work had studied negation in traditional retrieval, but little work had directly evaluated negation in modern neural IR models.

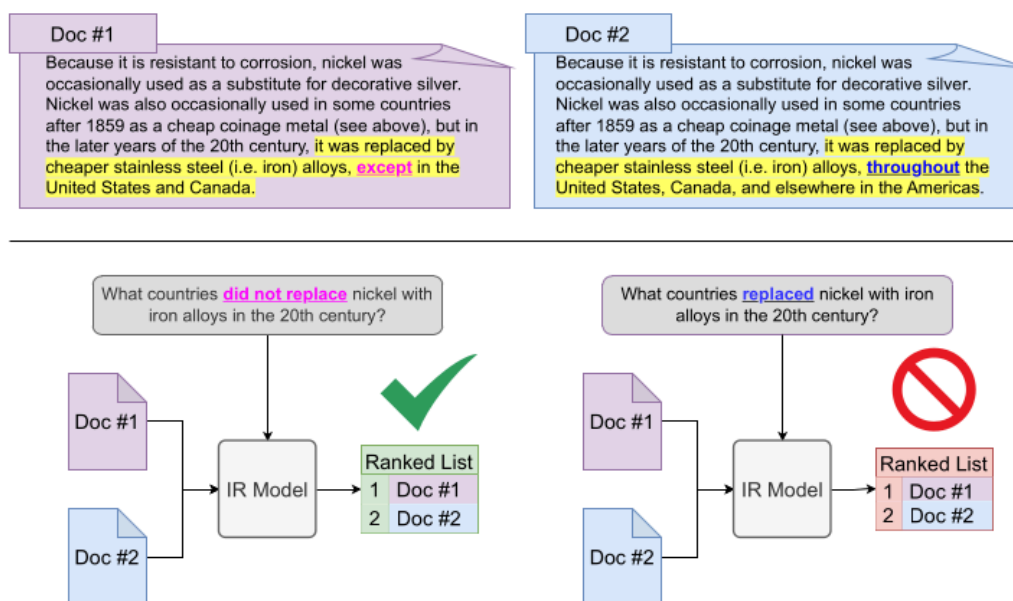
Overview of the Original Paper

Paper Objective

The paper proposes NevIR, a benchmark for evaluating negation in neural information retrieval. Instead of testing retrieval on a large document collection, NevIR uses contrastive document pairs. Each pair contains two nearly identical documents, but one differs from the other through a crucial negation or meaning reversal. The model must rank the correct document higher for both related queries.

Paper Objective

NevIR is built from CondaQA. The authors use document pairs that differ minimally but contain changes involving negation. They collect natural language queries using Mechanical Turk, where each pair has two queries: one relevant to document 1 and one relevant to document 2. The dataset has 948 training pairs, 225 development pairs, and 1,383 test pairs.



Each NevIR example contains:

Component	Meaning
q1	Query that should retrieve doc1
q2	Query that should retrieve doc2
doc1	First document
doc2	Contrastive document differing by negation or meaning reversal

Evaluation Metric Used in the Paper

The main metric is pairwise accuracy. A pair is counted as correct only if the model ranks the correct document higher for both queries. This is stricter than normal query-level accuracy because a model can get one query correct by chance or by always preferring one document. The paper uses this metric because many models tend to rank the same document higher for both the negated and non-negated query.

Pairwise Accuracy = No. of pairs where both q1 and q2 are correctly ranked / Total pairs

Original Paper Results

The original paper evaluates sparse models, late-interaction models, bi-encoders, and cross-encoders. The results show that most models perform worse than random ranking on the NevIR task. The expected random pairwise accuracy is 25%. TF-IDF scores only 2.0%, SPLADE models score around 8%, bi-encoders mostly remain below 12%, ColBERT models perform below random, and cross-encoders perform best. MonoT5 3B achieves the highest reported score, 50.6%, but this is still far below human performance.

Model Type	Example Model	Paper Pairwise Accuracy
Random	Random ranking	25.0%
Sparse	TF-IDF	2.0%
Late Interaction	ColBERTv1	19.7%
Bi-encoder	all-mpnet-base-v2	8.1%
Cross-encoder	MonoT5 base	34.9%

Cross-encoder	MonoT5 3B	50.6%
---------------	-----------	-------

The paper concludes that cross-encoders are better suited to negation-sensitive ranking because they jointly encode the query and document, while sparse models and bi-encoders often fail to distinguish the contrastive documents.

Our Implementation

Repository Overview

Our implementation, Negation-Lens, is a reproduction-style project for evaluating negation-aware information retrieval. The repository uses the NevIR dataset, where each row contains two queries and two documents. The intended mapping is that *q1* retrieves *doc1* and *q2* retrieves *doc2*. The implemented models include Random, BM25, TF-IDF, Sentence-BERT, and MonoT5. The repository also computes query-level accuracy, pairwise accuracy, and MRR.

Dataset Used

We used the NevIR dataset splits stored locally as *train.csv*, *validation.csv*, and *test.csv*. Each CSV contains the columns *id*, *WorkerId*, *q1*, *q2*, *doc1*, and *doc2*. For every row, *q1* is expected to match *doc1*, while *q2* is expected to match *doc2*.

Models Implemented

Model	Type	Purpose
Random	Baseline	Checks chance-level performance
BM25	Sparse lexical retrieval	Tests traditional term-matching retrieval
TF-IDF	Sparse lexical retrieval	Tests vector-space lexical similarity
Sentence-BERT	Bi-encoder	Tests embedding-based semantic retrieval
MonoT5 3B	Cross-encoder / reranker	Tests joint query-document scoring

Experimental Setup

The evaluation was performed on the validation and test splits of NevIR. For each example, the model scores both documents for each query and ranks them. We then compute query-level

accuracy, pairwise accuracy, and MRR. The main comparison with the original paper is based on pairwise accuracy, because that is the primary metric used in NevIR.

Item	Value
Programming language	Python
Dataset	NevIR
Splits used	Validation and Test
Models	Random, BM25, TF-IDF, SBERT, MonoT5

```
python src/evaluate.py --split validation --model all
python src/evaluate.py --split test --model all
python src/report.py
```

Evaluation Metrics

Query Accuracy

Query accuracy measures whether each individual query retrieves its correct document. Since each row has two queries, this metric is calculated over all query cases.

Pairwise Accuracy

Pairwise accuracy is the main metric. A pair is correct only if both queries in the same row are correctly ranked. This is stricter than query accuracy and better reflects whether the model understands negation.

Mean Reciprocal Rank

MRR measures the average reciprocal rank of the correct document. Since each query has only two candidate documents, the reciprocal rank is 1 if the correct document is ranked first and 0.5 if it is ranked second.

Our Results

Validation Results

Model	Query Accuracy	Pairwise Accuracy	MRR	Query Cases
Random	0.509	0.249	0.754	450
BM25	0.491	0.022	0.746	450
TF-IDF	0.502	0.062	0.751	450
SBERT	0.522	0.084	0.761	450
MonoT5	0.691	0.422	0.846	450

Test Results

Model	Query Accuracy	Pairwise Accuracy	MRR	Query Cases
Random	0.492	0.244	0.746	2766
BM25	0.484	0.018	0.742	2766
TF-IDF	0.484	0.043	0.742	2766
SBERT	0.523	0.071	0.762	2766
MonoT5	0.722	0.477	0.861	2766

Comparison With the Original Paper

Model	Paper Pairwise Accuracy	Our Test Pairwise Accuracy	Difference	Comment
Random	25.0%	24.4%	-0.6%	Very close; random varies slightly due to sampling

BM25	Not reported	1.8%	N/A	Paper excluded BM25 because it behaves similarly to TF-IDF on this small paired setup
TF-IDF	2.0%	4.3%	+2.3%	Similar failure trend, but exact value differs
SBERT / Bi-encoder	8.1% for all-mpnet-base-v2	7.1% for all-MiniLM-L6-v2	Not exact model match	Same model family, different checkpoint
MonoT5 3B	50.6%	47.7%	-2.9%	Close trend, but not exact reproduction

Our results do not exactly match the original paper, but they reproduce the main trend. Sparse retrieval methods such as BM25 and TF-IDF perform far below random pairwise accuracy. Sentence-BERT also performs poorly, showing that embedding similarity alone is not sufficient for negation understanding. MonoT5 performs the best, which agrees with the paper’s conclusion that cross-encoders are more effective because they jointly process the query and document.

Why Our Results Differ From the Paper

Different Model Coverage

The paper evaluates many models, including SPLADE, ColBERT, several bi-encoders, and multiple MonoT5 sizes. Our implementation evaluates a smaller set: Random, BM25, TF-IDF, SBERT, and MonoT5.

Different Checkpoints

The paper reports results for models such as all-mpnet-base-v2, multi-qa-mpnet-base-dot-v1, and MonoT5 variants. Our SBERT model uses all-MiniLM-L6-v2, so it is not directly identical to the paper’s bi-encoder results.

Preprocessing Differences

TF-IDF and BM25 results can change depending on tokenization, lowercasing, punctuation handling, stopword removal, and tie-breaking.

Tie Handling

In a two-document ranking setup, if both documents receive the same or very close scores, the ordering rule can affect pairwise accuracy.

Metric Differences

The paper’s main table reports pairwise accuracy. Our implementation also reports query accuracy and MRR. These should not be directly compared with the paper’s pairwise accuracy.

Dataset Split Naming/Version Differences

The paper uses train/dev/test terminology, while our repository uses train/validation/test. Even small differences in dataset source or preprocessing can affect exact reproduction.

MonoT5 Implementation Details

MonoT5 scores can differ because of checkpoint version, input formatting, truncation length, batching, precision, and model-loading pipeline.

Therefore, our reproduction should be considered a partial reproduction rather than an exact one. The numerical values differ, but the core conclusion remains consistent with the paper: models that rely on lexical overlap or independent sentence embeddings struggle with negation, while cross-encoder models perform better.

Error Analysis

We analyzed cases where models failed to rank the correct document. BM25 and TF-IDF often fail because the two documents share almost the same vocabulary. SBERT fails because the embeddings of negated and non-negated sentences remain highly similar. MonoT5 performs better but still fails on some examples where the negation is implicit, comparative, or expressed through subtle wording changes.

Error Type	Description
Always ranks doc1 higher	Model ignores query difference

Always ranks doc2 higher	Model has document-position bias
Lexical overlap failure	Model matches shared words only
Semantic similarity failure	SBERT treats contrastive docs as similar
Subtle negation failure	Model misses “except,” “without,” “less than,” etc.

Discussion

The results show that negation remains a difficult problem for information retrieval systems. Lexical models fail because they cannot understand the meaning of negation. Bi-encoders such as Sentence-BERT are better at semantic similarity but still encode the query and document separately, which makes them weak at distinguishing small meaning reversals. Cross-encoders such as MonoT5 perform better because they compare the query and document jointly, but even MonoT5 does not reach human-level performance.

The original paper also found that fine-tuning on NevIR improves performance, but still leaves a gap to perfect accuracy and may hurt performance on traditional benchmarks such as MS MARCO.

Limitations

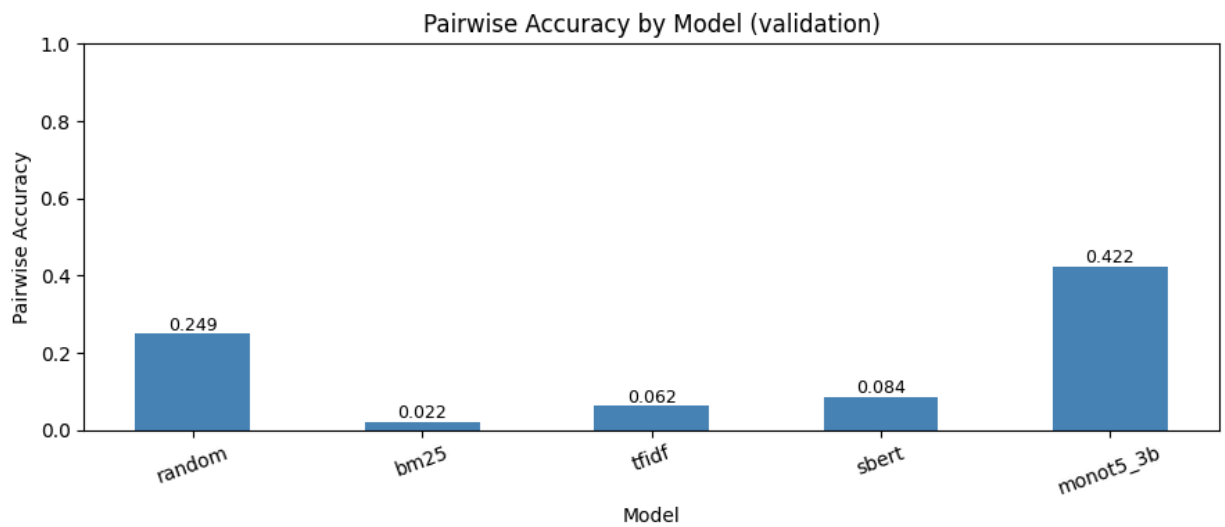
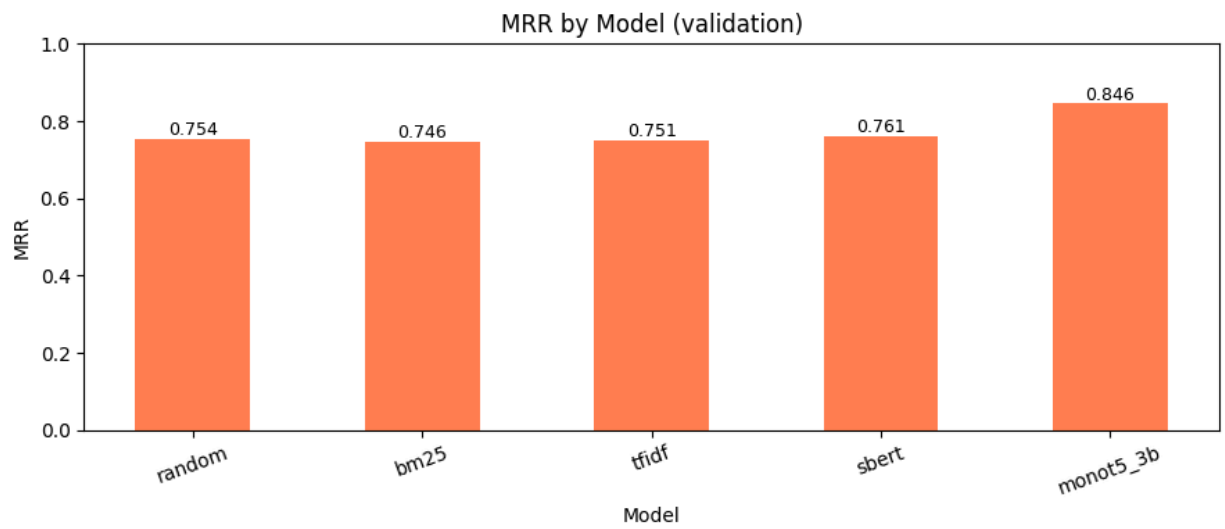
Our reproduction has several limitations. First, we did not implement all models evaluated in the original paper, such as SPLADE, ColBERT, RocketQA, and all MonoT5 variants. Second, our SBERT checkpoint differs from the paper’s reported bi-encoder checkpoints, so the comparison is not exact. Third, we did not reproduce the fine-tuning experiments from the paper. Fourth, preprocessing and tie-breaking details may differ from the original implementation. Finally, our negation type labels are rule-based and may not perfectly capture all forms of negation.

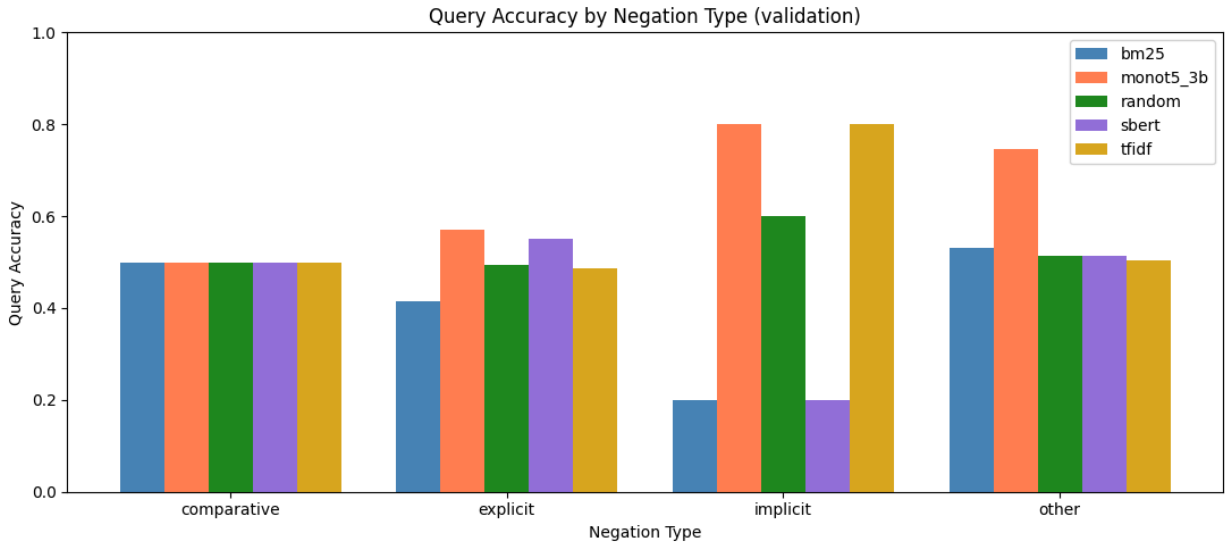
Conclusion

In this project, we reproduced the main NevIR evaluation pipeline and compared our results with the original paper. Although our exact numerical results differ from the paper, the overall trend is consistent. BM25, TF-IDF, and Sentence-BERT perform poorly on negation-sensitive retrieval, while MonoT5 performs significantly better. This supports the paper’s main claim that current retrieval models often fail to properly handle negation. Our implementation demonstrates that negation remains a challenging problem in neural information retrieval and that more robust training, better architectures, and targeted fine-tuning are needed.

Figures

On Validation Split





On Test Split



